

第 3 章：Prompt、Agent 与系统评测基础

3.1 本章符号说明

符号/缩写	English	中文	含义
Prompt	Prompt	提示词	给模型的任务描述、约束和上下文。
CoT	Chain-of-Thought	思维链	让模型生成或内部使用中间推理步骤。
RAG	Retrieval-Augmented Generation	检索增强生成	检索外部文档后再生成答案。
Agent	Agent	智能体	能规划、调用工具、观察结果并继续行动的系统。
Tool Use	Tool Use	工具调用	模型调用搜索、数据库、代码解释器等外部工具。
Function Calling	Function Calling	函数调用	模型输出结构化调用参数，由系统执行函数。
API	Application Programming Interface	应用程序编程接口	工具和外部服务的调用接口。
DB	Database	数据库	RAG 或工具调用中常访问的数据系统。
GUI	Graphical User Interface	图形用户界面	OSWorld / computer-use 类 agent 操作的界面。
MCP	Model Context Protocol	模型上下文协议	暴露 tools、resources、prompts 的上下文协议范式。
Memory	Memory	记忆	agent 跨轮次保存的任务状态、用户偏好或外部事实。
Planner	Planner	规划器	负责拆任务、排步骤、选择工具的模块或行为。
Executor	Executor	执行器	负责调用工具、运行代码、修改文件等动作的模块。
Verifier	Verifier	验证器	检查答案、代码、引用或最终状态是否正确的模块。

符号/缩写	English	中文	含义
Embedding	Embedding	向量表示	文本、图像或多模态对象的语义向量。
BM25	Best Matching 25	BM25 排序函数	经典关键词检索打分方法。
Reranker	Reranker	重排序模型	对召回文档重新排序，提高证据质量。
MRR	Mean Reciprocal Rank	平均倒数排名	检索排序评测指标。
recall@k	Recall at k	前 k 召回率	gold evidence 是否出现在前 k 个结果中。
nDCG	Normalized Discounted Cumulative Gain	归一化折损累计增益	检索排序质量指标。
MTEB	Massive Text Embedding Benchmark	大规模文本向量评测	embedding 模型评测基准。
HELM	Holistic Evaluation of Language Models	语言模型整体评测	覆盖准确率、公平性、鲁棒性等维度的评测框架。
AIME	American Invitational Mathematics Examination	美国邀请数学竞赛	数学推理评测。
GPQA	Graduate-Level Google-Proof Q&A	研究生级抗搜索问答	高难科学知识和推理评测。
SWE-bench	Software Engineering Benchmark	软件工程基准	真实 GitHub issue 修复任务。
TAU-bench	Tool-Agent-User Benchmark	工具-智能体-用户交互基准	业务 API 多轮交互任务。
OSWorld	Operating System World Benchmark	操作系统世界基准	GUI/computer-use 任务评测。
PII	Personally Identifiable Information	个人可识别信息	安全评测中需要防泄漏的敏感信息。
RAGAS	RAG Assessment	RAG 评估工具	RAG 自动评测工具，详见第 10 章。
q	Query	查询	用户问题或检索查询。
D	Document Corpus	文档库	可检索的外部知识集合。
d_i	Retrieved Document	检索文档	被召回的第 i 个文档块。

3.2 本章目标

- 掌握 prompt engineering 的边界。
- 理解基础 RAG pipeline；高级 RAG、Agentic RAG 和 RAG eval 在第 10 章展开。
- 理解 agentic tool use 的系统结构。
- 能设计 LLM 应用的离线评测、在线监控和安全边界。

3.3 Prompt Engineering 的边界

好的 prompt 通常包含：

- Role：模型扮演什么角色。
- Task：具体要完成什么。
- Context：可用信息和背景。
- Constraints：格式、风格、禁止事项。
- Examples：few-shot 示例。
- Evaluation criteria：怎样算好。

面试表达：

Prompt engineering 不是玄学调参，而是在模型上下文里显式提供任务定义、约束、示例和输出格式。它能提升 instruction following，但不能从根本上补齐模型没有的知识、工具、权限和验证闭环。

3.4 CoT、Hidden Reasoning 与 Thinking Budget

CoT 可以提升复杂推理任务表现，但 2026 年面试要区分三件事：

1. 用户可见的解释：给用户看的推理摘要。
2. 模型内部的 reasoning trace：可能不展示，用于内部决策。
3. thinking budget：允许模型消耗多少推理 token 或时间。

工程注意点：

- 简单任务强行 CoT 会增加延迟和幻觉。
- 安全/隐私任务不应该暴露完整内部推理。
- reasoning token 越多不一定越好，要看任务是否可验证。
- 对 agent 来说，工具观察比纯文本推理更可靠。

3.5 RAG 基础 Pipeline

本节只建立基础概念。生产级 RAG、GraphRAG、Agentic RAG、RAGAS、权限和引用评测在第 10 章系统展开。

典型 RAG：

1. 文档解析和清洗。
2. chunking。
3. embedding。
4. 建立向量索引。

- 5. query rewrite。
- 6. retrieval。
- 7. reranking。
- 8. context packing。
- 9. generation。
- 0. citation 和 answer verification。

可以写成：

$$q \rightarrow \{d_1, \dots, d_k\} \rightarrow LLM(q, d_1, \dots, d_k)$$

RAG 解决的问题：

- 知识更新。
- 私有知识接入。
- 引用溯源。
- 降低纯参数记忆导致的幻觉。

RAG 解决不了的问题：

- 检索不到就无法回答。
- 检索到错误文档会污染生成。
- 长上下文里模型可能忽略关键证据。
- 权限、隐私、数据新鲜度仍需系统设计。

3.6 2026 RAG 重点：不是只做向量库

面试里不要把 RAG 说成“embedding + vector DB”。更完整的路线包括：

模块	English	中文	面试关注点
Hybrid Retrieval	Hybrid Retrieval	混合召回	BM25 + dense embedding，兼顾关键词和语义。
Query Rewrite	Query Rewrite	查询改写	多轮问题、省略指代、领域术语转换。
Reranking	Reranking	重排序	提高 top-k 证据相关性。
Context Packing	Context Packing	上下文打包	控制长度、去重、保留表格结构。
Citation Check	Citation Verification	引用校验	答案是否真正被引用支持。
Permission Filter	Permission Filtering	权限过滤	用户只能检索有权限文档。
Freshness	Freshness Control	新鲜度控制	数据版本、过期文档、增量索引。

长上下文模型并不自动替代 RAG。长上下文适合一次性给大量材料，RAG 适合动态知识库、权限隔离、引用和成本控制。现实系统常用 long-context RAG：先检索，再把更长证据包交给模型。

3.7 Agent 与 ReAct

Agent 不是“模型变聪明了”这么简单，而是系统让模型进入循环：

```
observe -> plan -> act/tool call -> observe -> verify -> continue/finish
```

ReAct 的思想是把 reasoning 和 action 交替起来，让模型一边推理一边调用外部工具。

工程上要关注：

- 工具 schema 是否清晰。
- 失败是否可恢复。
- 是否有循环上限。
- 是否记录轨迹用于 debug。
- 是否有权限和安全边界。
- 是否有 verifier 检查最终状态，而不是只看自然语言回答。

3.8 Tool Use：从 Function Calling 到 Agentic Tool Use

普通 function calling：

- 用户请求明确。
- 模型选择一个函数。
- 系统执行函数。
- 模型总结结果。

agentic tool use：

- 模型需要自己拆任务。
- 多工具、多轮调用。
- 工具结果会改变下一步计划。
- 失败时要重试、换策略或向用户澄清。
- 任务结束前要验证最终状态。

面试表达：

Function calling 是结构化输出能力，agentic tool use 是多步决策能力。真正难的是任务分解、状态管理、错误恢复、权限控制和结果验证。

3.9 Agent 系统结构

一个比较稳的 agent 系统通常包含：

模块	English	中文	作用
Controller	Controller	控制器	管理循环、预算、终止条件。
Planner	Planner	规划器	拆解任务、选择工具。
Tool Runtime	Tool Runtime	工具运行时	执行搜索、代码、数据库、浏览器、文件操作。
Memory Store	Memory Store	记忆存储	保存长期偏好、任务状态、轨迹摘要。
Verifier	Verifier	验证器	检查测试、引用、状态、格式。
Policy Guard	Policy Guard	策略防护	权限、安全、隐私、速率限制。
Trace Logger	Trace Logger	轨迹日志	记录每次 tool call，便于 debug 和 eval。

不要在面试里说“我们用了 LangChain 所以是 agent”。框架不是核心，核心是任务闭环和评测闭环。

3.10 Evaluation：从 Model Eval 到 System Eval

LLM 应用不能只看 benchmark，需要分层评测。

层级	English	中文	例子
Model Eval	Model Evaluation	模型能力评测	MMLU、GPQA、AIME、HumanEval。
Retrieval Eval	Retrieval Evaluation	检索评测	recall@k、MRR、nDCG。
Generation Eval	Generation Evaluation	生成评测	factuality、faithfulness、format。
Tool Eval	Tool Evaluation	工具评测	tool call accuracy、参数准确率、错误恢复率。
Agent Eval	Agent Evaluation	智能体评测	SWE-bench、TAU-bench、BrowseComp、OSWorld。
System Eval	System Evaluation	系统评测	端到端成功率、延迟、成本、拒答率。
Safety Eval	Safety Evaluation	安全评测	jailbreak、PII 泄露、prompt injection、权限绕过。

面试表达：

LLM 项目评测要把“模型能力”和“系统能力”拆开。RAG 场景中，答案错可能来自 query rewrite、embedding、chunk、retrieval、rerank、context packing 或 generation。Agent 场景中，错误还可能

来自 tool schema、状态管理、权限、预算和 verifier。

3.11 Prompt Injection 与 Tool Safety

Agent 接入工具后，安全问题会放大：

- 网页或文档里可能包含恶意指令。
- 检索内容可能要求模型泄露系统提示。
- 工具可能有写权限、删库权限、转账权限。
- 长任务中模型可能忘记原始约束。

基础防线：

1. 把用户指令、系统指令、工具输出分层。
2. 工具输出默认是不可信数据。
3. 高风险工具需要 human approval。
4. 每个工具做最小权限。
5. 对写操作做 dry-run 和 diff。
6. 记录 trace，便于审计。

3.12 本章面试高频问题

3.12.1 RAG 为什么能缓解幻觉？

RAG 把外部知识作为上下文提供给模型，让模型基于检索证据回答，而不是只依赖参数记忆。它还可以提供引用和可更新知识库。

3.12.2 RAG 的主要失败模式是什么？

主要包括检索不到、检索错、chunk 切坏、rerank 失败、上下文过长导致证据被忽略、文档冲突、权限过滤错误和生成阶段不忠实。

3.12.3 Agent 和普通 chat completion 有什么区别？

普通 chat completion 一次输入一次输出。Agent 多了工具调用、状态管理、观察反馈、多步决策、错误恢复和最终验证。

3.12.4 怎么评估一个企业知识库问答系统？

需要评估检索 recall、答案 faithfulness、引用准确率、拒答策略、权限隔离、延迟、成本和人工标注通过率。

3.12.5 怎么评估一个工具调用 agent？

不要只看最终回答。要看 tool selection accuracy、参数准确率、调用次数、成功率、错误恢复率、最终状态正确率、成本、延迟和安全违规率。

3.13 本章 References

- [Chain-of-Thought Prompting Elicits Reasoning in Large Language Models](#)

- [Self-Consistency Improves Chain of Thought Reasoning in Language Models](#)
- [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)
- [REALM: Retrieval-Augmented Language Model Pre-Training](#)
- [ReAct: Synergizing Reasoning and Acting in Language Models](#)
- [Toolformer: Language Models Can Teach Themselves to Use Tools](#)
- [AgentBench: Evaluating LLMs as Agents](#)
- [TAU-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains](#)
- [BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents](#)
- [Holistic Evaluation of Language Models](#)
- [MTEB: Massive Text Embedding Benchmark](#)

[章节索引](#)

[← 上一章](#) [下一章 →](#)
